Aula 07 — Express

Rotas e middlewares

Construindo uma cadeia HTTP organizada

Missão da aula

Construir e depurar uma cadeia Express

Organizar produtos com express.Router()

Ler parâmetros com req.params

Encadear middlewares com next()

Tratar 404 e erros globais

Fundamentos essenciais

Middleware

Função executada durante o caminho da requisição. Pode observar, alterar, responder ou encaminhar.

Ordem e next()

O Express percorre os componentes na ordem registrada. next entrega o controle à próxima etapa.

Desafios no Sandbox

Experimente a cadeia Express antes da API cumulativa

Preparação do Sandbox

```
node --version
mkdir exemplos\Exemplo00-Sandbox
cd exemplos\Exemplo00-Sandbox
npm init -y
npm install express

package.json
"type": "module"
```

Mini-desafio 1 — ordem das rotas (ordem.js)

```
const app = express();
const router = express.Router();

router.get('/:id', (req, res) =>
  res.json({ rota: 'id', valor: req.params.id })
);
router.get('/categorias', (_req, res) =>
  res.json({ rota: 'categorias' })
);
```

Mini-desafio 1 — prever e observar (ordem.js)

```
app.use('/api/produtos', router);
app.listen(3101, '127.0.0.1');

GET /api/produtos/categorias

Qual rota respondeu?

Mova /categorias para cima de /:id
Reinicie e repita
```

Mini-desafio 2 — params e query

```
router.get('/:id', (req, res) => {
  res.json({
    params: req.params,
    query: req.query
  });
});

GET /api/produtos/7?detalhes=sim
```

Mini-desafio 2 — body JSON

```
app.use(express.json());

router.post('/', (req, res) => {
  res.status(201).json({ body: req.body });
});

Postman: Body > raw > JSON
```

Mini-desafio 3 — papel de next()

```
app.use((req, _res, next) => {
  console.log('antes', req.method,
    req.originalUrl);
  next();
  console.log('depois', req.method,
    req.originalUrl);
});
```

Uma função precisa concluir o fluxo

Responder
`res.status(...).json(...)`

ou

Encaminhar
`next()`

Depois de responder, não chame `next()`

Mini-desafio 4 — medir até finish

```
app.use((req, res, next) => {
  const inicio = performance.now();
  res.once('finish', () => {
    console.log(req.method, req.originalUrl,
      res.statusCode,
      Math.round(performance.now() - inicio));
  });
  next();
});
```

Mini-desafio 5 — fallback 404

```
app.use((req, res) => {
  res.status(404).json({
    erro: `Rota ${req.method} ${req.originalUrl}
      não encontrada.`
  });
});
```

Mini-desafio 5 — tratador de erro

```
app.use((erro, _req, res, _next) => {
  const status =
    erro.type === 'entity.parse.failed' ? 400 : 500;
  const mensagem = status === 400
    ? 'JSON malformado.'
    : 'Erro interno do servidor.';
  res.status(status).json({ erro: mensagem });
});
```

Evolução hands-on

Da leitura passiva à construção da arquitetura

Acesse a API cumulativa

```
cd ..\api-produtos

Ponto de partida:
app.js contém GET e POST de produtos

Objetivo:
routes cuida dos endpoints
app.js coordena a cadeia
```

Passo 1 (app.js)

Criar o roteador de produtos

Crie a pasta e o arquivo

```
mkdir routes
type nul > routes\produtoRoutes.js

Abra:
routes/produtoRoutes.js

Transfira para ele a lógica de GET e POST
```

produtoRoutes.js — catálogo

```
import express from 'express';
import { resolve } from 'node:path';
import { criarCatalogoArquivo } from
  '../catalogo/catalogoArquivo.js';

const caminhoCatalogo =
  process.env.CATALOGO_ARQUIVO ||
  resolve(import.meta.dirname,
    '../data/produtos.json');
```

produtoRoutes.js — criar o Router

```
const catalogo =
  criarCatalogoArquivo(caminhoCatalogo);

export const produtoRoutes =
  express.Router();

// Dentro do Router, / representa
// o prefixo montado em app.js
```

Transfira GET para o Router

```
produtoRoutes.get('/',
  async (_req, res, next) => {
  try {
    const produtos = await catalogo.listar();
    res.status(200).json({
      sucesso: true, dados: produtos
    });
  } catch (erro) {
    next(erro);
  }
});
```

Transfira POST para o Router

```
produtoRoutes.post('/',
  async (req, res, next) => {
  try {
    const { nome, preco, estoque, categoria } =
      req.body;
    const produto = await catalogo.criar({
      nome, preco, estoque, categoria
    });
```

POST — criação e validação

```
    res.status(201).json({
      sucesso: true, dados: produto
    });
  } catch (erro) {
    res.status(400).json({
      sucesso: false, erro: erro.message
    });
  }
});
```

O Router usa caminhos relativos

SENAI

No módulo

produtoRoutes.get('/') e produtoRoutes.post('/') descrevem operações da coleção.

No app.js

app.use('/api/produtos', produtoRoutes) acrescenta o prefixo comum.

Passo 2

Adicionar uma rota com parâmetro

GET /:id — converter e validar

```
produtoRoutes.get('/:id',
  async (req, res, next) => {
  try {
    const id = Number(req.params.id);
    if (!Number.isInteger(id)) {
      return res.status(400).json({
        sucesso: false,
        erro: 'ID deve ser um número inteiro.'
      });
    }
```

GET /:id — consultar o catálogo

```
    const produto =
      await catalogo.buscarPorId(id);
    res.status(200).json({
      sucesso: true, dados: produto
    });
  } catch (erro) {
```

GET /:id — mapear ausência

```
    if (erro.message.includes('não encontrado')) {
      return res.status(404).json({
        sucesso: false,
        erro: erro.message
      });
    }
    next(erro);
  }
});
```

Três resultados da busca por ID

400 ou 404

ID inválido gera 400. ID válido sem produto gera 404 com a mensagem do catálogo.

200 ou next(erro)

Produto localizado gera 200. Falha inesperada segue para o tratador global.

Passo 3

Integrar rotas e middlewares no app.js

app.js — imports e parser

```
import express from 'express';
import { carregarAmbiente } from
  './config/ambiente.js';
import { produtoRoutes } from
  './routes/produtoRoutes.js';

const configuracao =
  carregarAmbiente('.env');
export const app = express();
app.use(express.json());
```

Middleware global — iniciar a medição

```
app.use((req, res, next) => {
  const inicio = performance.now();

  res.once('finish', () => {
    const duracao = Math.round(
      performance.now() - inicio
    );
```

Middleware global — registrar e avançar

```
    console.log(
      `${req.method} ${req.originalUrl} - ` +
      `${res.statusCode} [${duracao}ms]`
    );
  });

  next();
});
```

Montar saúde e produtos

```
app.get('/api/saude', (_req, res) =>
  res.status(200).json({ status: 'ok' })
);

app.use('/api/produtos', produtoRoutes);
```

Fallback 404 fica depois das rotas

```
app.use((req, res) => {
  res.status(404).json({
    sucesso: false,
    erro: `A rota ${req.method} ` +
      `${req.originalUrl} não existe.`
  });
});
```

Tratador global fica no rodapé

```
app.use((erro, req, res, _next) => {
  console.error('Erro Capturado:', erro.message);
  res.status(500).json({
    sucesso: false,
    erro: 'Falha interna do servidor.'
  });
});
```

A ordem define o comportamento

```
1  express.json interpreta o body
2  medidor inicia e chama next
3  saúde ou produtoRoutes tenta responder
4  fallback atende rota inexistente
5  tratador recebe next(erro)

Mover o fallback para cima bloqueia as rotas
```

Passo 4

Testar a arquitetura no Postman

Inicie o servidor

```
npm start

Mantenha o terminal visível

O medidor registrará:
método + caminho + status + duração
```

Evidências 1 e 2 — sucesso

```
GET http://localhost:3000/api/produtos
Esperado: 200 + coleção em dados
Terminal: tempo de resposta

GET http://localhost:3000/api/produtos/2
Esperado: 200 + produto do ID 2
```

Evidência 3 — parâmetro inválido

```
GET http://localhost:3000/api/produtos/abc

Esperado: 400

ID deve ser um número inteiro.
```

Evidência 4 — recurso ausente

```
GET http://localhost:3000/api/produtos/999

Esperado: 404

Mensagem de produto não encontrado
produzida pelo catálogo
```

Evidência 5 — fallback genérico

```
GET http://localhost:3000/rota-maluca

Esperado: 404

A rota GET /rota-maluca não existe.
```

Compare os dois 404

Recurso ausente

/api/produtos/999 corresponde ao Router, mas o catálogo não localiza o produto.

Rota inexistente

/rota-maluca não corresponde a nenhum endpoint e chega ao fallback genérico.

Laboratório de diagnóstico

```
req.body indefinido → mover express.json
/categorias vira ID → rota literal primeiro
requisição pendente → responder ou next()
headers enviados → resposta seguida de next
erro retorna HTML → conferir 4 parâmetros
404 vira 500 → diferenciar rota e exceção
```

Fechamento prático

Aponte no código real:

onde chegam params, query e body
onde o fluxo continua com next
onde nasce o fallback 404
onde um erro vira resposta 500

Encerre o servidor com Ctrl+C